QML Journal Club

Choong Pak Shen

March 15, 2026

QML Journal Club 1st meeting

Motivation

Quantum machine learning looks at the synergies between machine learning and quantum information, or how quantum computing may assist machine learning due to its different computing architecture.

Are quantum computers better at machine learning than classical computers? It depends.

1. Structure of data
2. Supervised machine learning or unsupervised machine learning
3. Quantum computing as a subroutine or optimizer
4. Types of quantum computer (annealer, superconducting qubit, continuous-variable systems)
5. Noise

What is quantum computing?

Quantum computer is a computing architecture that makes use of quantum physics to manipulate information.

The first quantum revolution happened around 20th-century, where classical physics was shown to be inadequate to explain several physical phenomena, such as blackbody radiation, photoelectric effect, and wave-particle duality.

The second quantum revolution started around 2000, where we have more matured technologies to control microscopic systems in a more precised manner, such that various quantum effects may be used in information processing.

These quantum systems are fragile and noisy. We are currently in the noisy-intermediate scale quantum (NISQ) era and moving towards fault-tolerant quantum era.

Characteristics of NISQ include:

1. Around 10-100 quantum bits (qubits)
2. No error correction

There are two types of algorithms,

1. Quantum algorithms: Algorithms that run on real quantum computers and produce theoretically proven quantum advantage. Current quantum hardware cannot implement these algorithms effectively.
2. NISQ algorithms: Algorithms that run on the current generation of quantum computers.

Quantum advantage

The holy grail of "quantum advantage"

1. Slower growth or asymptotic runtime with the size of dataset compared with a known machine learning algorithm?
2. Provable speedup?
3. Benchmarking against the best known classical algorithm, or computational complexity analysis?
4. Energy consumption?
5. The precision of output?
6. Probability of computational failure?
7. How much data to train a model?
8. How well can the model generalizes to different situations?

Claims of exponentially fast quantum machine learning algorithms come with a lot of assumptions.

Quantum machine learning

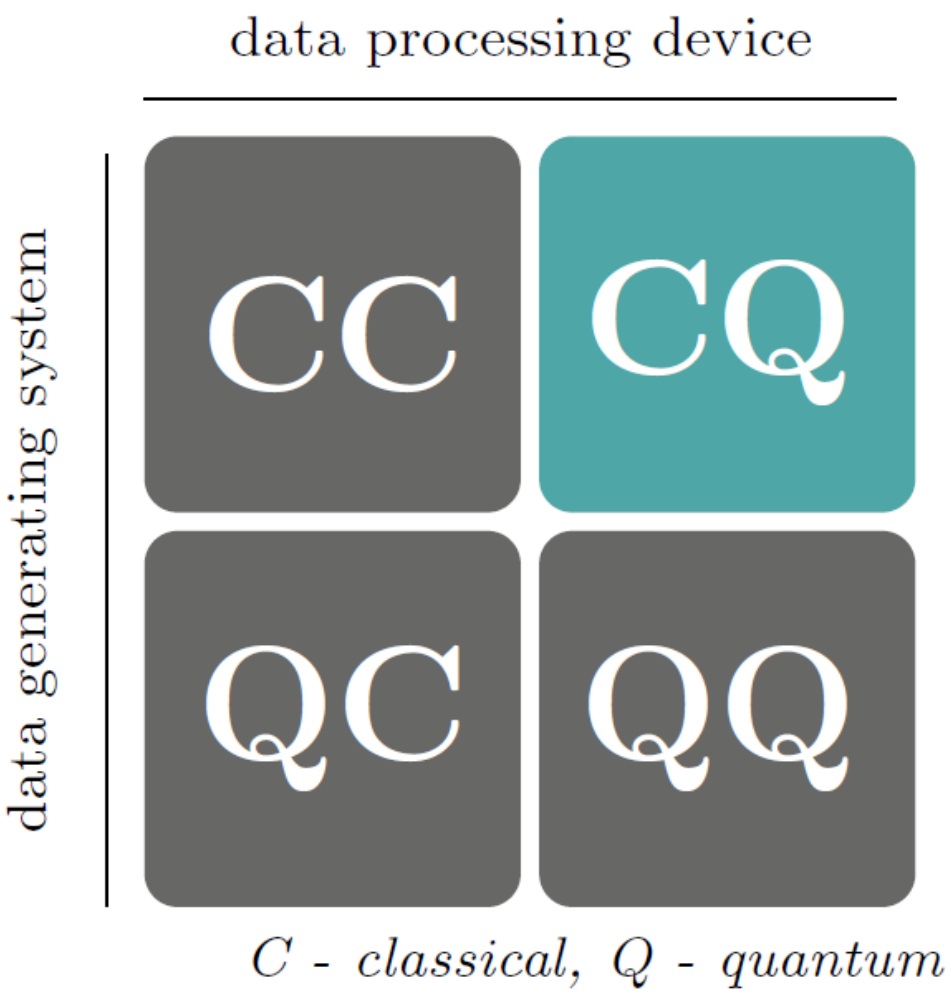

Figure 1: Four approaches to combine quantum computing and machine learning.

CC refers to classical data being processed classically. Examples would be the application of tensor networks, which was originally developed for quantum many-body systems to neural network training, or quantum-inspired machine learning models.

QC refers to quantum data being processed classically. Examples would be to use neural networks to discriminate quantum states or phase transitions in quantum many-body systems.

CQ refers to classical data being processed using quantum devices, and it is the scope of quantum machine learning. Examples would be to use time series data, satellite images or numerical data on quantum computers for analysis. Note that the data has to be loaded onto a quantum computer, which will be proportional to the data size.

QQ refers to quantum data being processed using quantum devices. Data could be derived from measuring a quantum system in an experiment, and then being fed back into a quantum computer, or quantum states. For an exponential speedup, quantum computer can access to quantum states (as data) directly.

Toy example: Nearest-neighbor classifier

Suppose you have two data points from the passenger list of Titanic, consisting of the price of their ticket (0 to 10,000), their cabin number (1 to 2500), and their survival. Given the same data point of a third passenger, can we identify the probability of survival for this third passenger?

	Raw data		Preprocessed data		
	Price	Cabin	Price	Cabin	Survival (y)
Passenger 1	8,500	0910	0.85	0.36	1 (yes)
Passenger 2	1,200	2105	0.12	0.84	0 (No)
Passenger 3	7,800	1121	0.78	0.45	?

The nearest-neighbor (squared-distance) classifier defines a closeness measure between the data points and the new input,

$$p(y=1|\vec{x}) = \frac{1}{\chi}\frac{1}{M}\sum_{m|y=1}\left(1-\frac{1}{c}\left\|\vec{x}-\vec{x}_m\right\|^2\right), \tag{1}$$

and it is based on the assumption that similar inputs should have a similar output.

Quantum squared-distance classifier

Step 1: Normalizing the length of each input vector to 1.

Passenger 1: $0.85^2 + 0.36^2 = 0.8521$, $\sqrt{\frac{0.85^2}{0.8521}} = 0.921$, $\sqrt{\frac{0.36^2}{0.8521}} = 0.39$

Passenger 2: $0.12^2 + 0.84^2 = 0.72$, $\sqrt{\frac{0.12^2}{0.72}} = 0.141$, $\sqrt{\frac{0.84^2}{0.72}} = 0.99$

Passenger 3: $0.78^2 + 0.45^2 = 0.8109$, $\sqrt{\frac{0.78^2}{0.8109}} = 0.866$, $\sqrt{\frac{0.45^2}{0.8109}} = 0.50$

Step 2: Data encoding by amplitude encoding.

Number of features: 6

Number of class labels: 2

Initial state is given by 4 qubits,

$$
\begin{aligned}
|\psi\rangle = \frac{1}{\sqrt{4}}([0.921|000\rangle + 0.39|001\rangle] \otimes |1\rangle + [0.141|010\rangle + 0.99|011\rangle] \otimes |0\rangle \\
+ [0.866|100\rangle + 0.50|101\rangle] \otimes |1\rangle + [0.866|110\rangle + 0.50|111\rangle] \otimes |0\rangle) \\
= \frac{1}{\sqrt{4}}(0.921|0001\rangle + 0.39|0011\rangle + 0.141|0100\rangle + 0.99|0110\rangle \\
+ 0.866|1001\rangle + 0.50|1011\rangle + 0.866|1100\rangle + 0.50|1110\rangle)
\end{aligned}
$$

Step 3: Apply Hadamard gate on the first qubit.

$$
\begin{aligned}
|\psi'\rangle = H_1|\psi\rangle = \frac{1}{\sqrt{8}}([0.921 + 0.866]|0001\rangle + [0.39 + 0.5]|0011\rangle \\
+ [0.141 + 0.866]|0100\rangle + [0.99 + 0.5]|0110\rangle \\
+ [0.921 - 0.866]|1001\rangle + [0.39 - 0.50]|1011\rangle \\
+ [0.141 - 0.866]|1100\rangle + [0.99 - 0.50]|1110\rangle)
\end{aligned}
$$

Step 4: Measure the first qubit and select samples (post-selection process) with $|q_1\rangle = |0\rangle$.

$$
\begin{aligned}
|\psi''\rangle = (M_1 + \mathrm{PS})|\psi'\rangle = \frac{1}{\sqrt{8\chi}}([0.921 + 0.866]|0001\rangle + [0.39 + 0.5]|0011\rangle \\
+ [0.141 + 0.866]|0100\rangle + [0.99 + 0.5]|0110\rangle)
\end{aligned}
$$

where χ is a normalization constant,

$$
\begin{aligned}
\chi &= \frac{1}{8}(|0.921 + 0.866|^2 + |0.39 + 0.5|^2 + |0.141 + 0.866|^2 + |0.99 + 0.5|^2) \\
&= 0.902
\end{aligned}
$$

Step 5: Measure the last qubit.

$$p(|q_4\rangle = |0\rangle) = 0.448$$
$$p(|q_4\rangle = |1\rangle) = 0.552$$

Alternatively, Step 4 and 5 can be combined by calculating the conditional probability,

$$\begin{aligned}
&p(|q_4\rangle = |0\rangle || q_1\rangle = |0\rangle) \\
&\quad = \frac{p(|q_4\rangle = |0\rangle) \cap p(|q_1\rangle = |0\rangle)}{p(|q_1\rangle = |0\rangle)} \\
&\quad = \frac{(0.141 + 0.866)^2 + (0.99 + 0.5)^2}{(0.921 + 0.866)^2 + (0.39 + 0.5)^2 + (0.141 + 0.866)^2 + (0.99 + 0.5)^2} \\
&\quad = 0.448
\end{aligned}$$

Let

$$a^2 + b^2 = 0.141^2 + 0.99^2 = 1,$$
$$c^2 + d^2 = 0.866^2 + 0.5^2 = 1.$$

We observe that

$$\begin{aligned}
p(|q_4\rangle = |0\rangle) &= \frac{1}{8\chi}[(0.141 + 0.866)^2 + (0.99 + 0.5)^2] \\
&= \frac{1}{8\chi}[(a + c)^2 + (b + d)^2] \\
&= \frac{1}{8\chi}[a + 2ac + c^2 + b^2 + 2bd + d^2] \\
&= \frac{1}{8\chi}[2 + 2ac + 2bd] = \frac{1}{4\chi}[1 + 1ac + 1bd] \\
&= \frac{1}{4\chi}[2 - (1 - ac - bd)] \\
&= \frac{1}{4\chi}[2 - \frac{1}{2}(a^2 + b^2 + c^2 + d^2 - 2ac - 2bd)]
\end{aligned}$$

$$
\begin{aligned}
p(|q_4\rangle = |0\rangle) &= \frac{1}{4\chi}[2 - \frac{1}{2}(\{a-c\}^2 + \{b-d\}^2)] \\
&= \frac{1}{2\chi}[1 - \frac{1}{4}(\{a-c\}^2 + \{b-d\}^2)] \\
&= \frac{1}{2\chi}[1 - \frac{1}{4}([0.141 - 0.866]^2 + [0.99 - 0.5]^2)]
\end{aligned}
$$

We see that this is the same as

$$
p(y=1|\vec{x}) = \frac{1}{\chi}\frac{1}{M}\sum_{m|y=1}\left(1 - \frac{1}{c}\left\|\vec{x} - \vec{x}_m\right\|^2\right).
$$

Insights from the toy example

The mathematical formalism of quantum theory provides an equivalent counterpart to machine learning. To some extent, quantum machine learning algorithms are often inspired by classical machine learning algorithms, and vice versa.

Data encoding is the most crucial step of quantum machine learning with classical data (CQ).

Quantum machine learning algorithms often requires further preprocessing on classical data.

The result of a quantum machine learning algorithm always require measurement.